

Cowboy Storage: CBFS and Runner Attached Storage

Technical Whitepaper

Charles DePue, Martin Aceto

April 2026 — Draft v0.9.0

1 Abstract

This document specifies Cowboy’s storage layer: the **Cowboy Blockchain File System (CBFS)** and **Runner Attached Storage (RAS)**. CBFS is a decentralized, encrypted, erasure-coded storage substrate operated by a permissionless network of **Relay Nodes**. RAS is the Cowboy protocol layer that mounts CBFS under account-scoped **Volumes**, anchors volume state on-chain via compact **StorageCommitments**, and mediates access to volumes through cryptographic **capability tokens** (CapTokens).

Together, CBFS and RAS provide actors and runners with a durable, integrity-protected, privacy-preserving storage primitive that (1) keeps the data path entirely off-chain — a read or write never costs a transaction — while (2) providing on-chain anchors sufficient for commitment, billing, repair, challenge, and eviction. A **delegation keypair** model separates cold wallet authority from hot storage authority, allowing owner-direct data operations to be authorized locally and verified offline by Relay Nodes against cached chain state.

This specification covers the volume lifecycle, CapToken and delegation cryptography, Reed-Solomon erasure coding, Relay Node registration and repair, Proof-of-Retrievability challenges, per-epoch storage rent, and all implementation parameters. It is a complement to the [Cowboy Technical Whitepaper](#), which is the normative reference for consensus, execution, economics, and actor semantics. Where this document and the Technical Whitepaper conflict, the Technical Whitepaper prevails.

2 Introduction

Actors on Cowboy are autonomous Python programs with on-chain storage subject to state rent and a 1 MiB default quota (extendable to 8 MiB via a storage bond) — see the Technical Whitepaper §3.2, §4.4. That model is sized for state, not for data. Modern autonomous agents need to persist working memory, cache retrieval traces, hold model adapters, stage artifacts between jobs, and serve large public assets. None of those fit naturally into on-chain state: they are too large, too churn-heavy, or too privacy-sensitive to push into a consensus-critical key/value store.

Cowboy’s answer is to separate two concerns that Ethereum fuses together:

- **State** — small, consensus-critical key/value data that participates directly in the state transition function. Billed per cell, subject to rent, stored in QMDB, and bounded by per-actor quotas.
- **Storage** — large, potentially private data, durably kept across a permissionless Relay Node network and referenced on-chain only by compact commitments. Billed per byte per

epoch in a separate market, mediated by capability tokens rather than transactions.

CBFS is the storage substrate. It is designed to be usable standalone (as a plain distributed filesystem) and as the data plane for Cowboy, with the same binaries and wire protocol in both configurations. RAS is the control plane: it defines the rules under which Cowboy accounts own volumes, grant runners access, settle storage rent, and register and slash Relay Nodes.

For architectural rationale and design decisions, see the [Cowboy Design Decisions Overview](#). For the on-chain specification from which this document is derived, see **CIP-9: CBFS-Backed Runner Storage**.

3 Architectural Overview

This section is descriptive and non-binding. Normative requirements are in §§1–13.

3.1 Terminology

- **Volume:** A named, account-scoped storage namespace. Each volume is a private (encrypted) or public (plaintext) container of objects identified by path keys.
- **Object:** A single blob within a volume, identified by its path. Encrypted client-side for private volumes; plaintext for public ones.
- **Relay Node:** A network participant that stores erasure-coded shards of volume data, answers reads, participates in repair, and serves Proof-of-Retrievability challenges. A distinct role from Validators and Runners.
- **Shard:** A fragment of an erasure-coded object. An object is split into K data + M parity shards; any K reconstruct the ciphertext.
- **Manifest:** The authoritative, Merkle-committed index of a volume's contents. Encrypted for private volumes, plaintext for public ones, itself stored as erasure-coded shards on Relay Nodes.
- **StorageCommitment:** The on-chain record of a volume's existence, owner, manifest root, erasure parameters, size, billing state, and status.
- **PlacementRecord:** The off-chain, mutable record binding a shard ID to its assigned Relay Nodes. Replicated across assigned nodes; versioned with compare-and-swap semantics.
- **CapToken:** A signed bearer token granting a specific runner or owner scoped (volume, path prefix, access mode, byte quota, time window) access to a volume. Two variants: runner CapTokens (issued by the chain at job dispatch) and owner CapTokens (client-signed by a delegated key).

- **DelegationCert:** A wallet-signed certificate that binds an Ed25519 CBFS key to a Cowboy account, scoped to storage use only, with a short (30-day) expiry.
- **Storage Manager (0x0A):** The Cowboy system actor that maintains StorageCommitments, mints runner CapTokens, handles per-epoch billing, and governs volume status transitions.
- **Relay Registry (0x0B):** The Cowboy system actor that maintains Relay Node profiles, staking, health, and repair/challenge coordination.

3.2 Key Features

CBFS and RAS together provide five core technical features:

- **Off-chain data plane.** Reads and writes are Relay-to-client RPCs authenticated by CapTokens; no on-chain transaction is submitted on the data path. The chain is touched only at volume lifecycle boundaries (create, delete, commit manifest root, register relay, settle billing, resolve challenge).
- **Client-side encryption and erasure coding.** Private volumes encrypt every object with AES-256-GCM using a per-volume Data Encryption Key (DEK), and split the ciphertext into K+M Reed-Solomon shards before distribution. Relay Nodes hold only ciphertext shards; they never see plaintext.
- **Delegation-based owner authorization.** A one-time wallet signature produces a short-lived DelegationCert binding an Ed25519 CBFS key to the account. The owner then constructs client-signed CapTokens locally for each data operation, with no further chain writes and no centralized token minter on the data path.
- **On-chain anchoring with off-chain serving.** A volume's manifest is served by Relay Nodes, but its 32-byte Merkle root is committed on-chain in the StorageCommitment. This yields READ_COMMITTED consistency across runners, supports Proof-of-Retrievability verification, and bounds the trust placed in any individual Relay Node.
- **Autonomous repair.** Relay Nodes continuously self-heal (verify local shards against expected hashes, reconstruct from peers) and coordinate dead-node replacement (leader-elected per shard via lowest-index rule, CAS-versioned PlacementRecords) without Runner involvement or per-event on-chain traffic.

3.3 Differences vs. On-chain State

Aspect	On-chain State	CBFS / RAS
Typical size	Bytes to kilobytes	Kilobytes to hundreds of GiB
Privacy	Public to all validators	Encrypted per volume (private mode)
Fees	Cells + state rent	Per-byte per-epoch storage rent, billed on-chain
Durability	Consensus replication across all validators	Reed-Solomon K+M across Relay Nodes
Access control	Address-derived ownership	CapTokens (path prefix, mode, TTL, quota)
Data path	Every access is a transaction	No on-chain transaction per access
Latency	Block time (~1 s)	Sub-second (network round trip)

4 Related Work

Decentralized storage has a decade of prior art. This section surveys the main systems and explains why none of them fit Cowboy’s target workload — swarms of autonomous Python actors dispatching jobs to short-lived runners that need private, mutable, POSIX-accessible storage at sub-second latency — and why CBFS and RAS are designed differently.

4.1 Content-Addressed Systems (IPFS, Swarm)

IPFS treats storage as immutable content addressed by a cryptographic hash (CID). Writing different bytes produces a different CID; mutation is expressed via an indirection layer (IPNS), which is slow and centralized in practice. IPFS has no inherent persistence: content disappears when its last pinner goes away. There is no economic layer, no access control beyond “know the CID,” and no SLA.

Swarm (Ethereum’s storage layer) adds an economic layer via **postage stamps** — upfront payments that buy storage for bounded horizons — and uses a forwarding Kademlia overlay over 4 KiB chunks. It remains content-addressed and immutable-by-default, with no per-object access control.

Content addressing is a poor fit for agent workloads:

- **Working state churns constantly.** Every minor write produces a new CID, forcing callers to rewrite pointers and invalidating caches.
- **No natural access scoping.** Privacy depends on not disclosing the CID; anyone who learns the CID can read forever.
- **Retrieval is best-effort.** Without pinning or gateways, there is no guarantee anyone is still serving the content.

CBFS instead uses **mutable, path-addressed volumes** with a Merkle-committed manifest. A write to `/foo/bar` replaces the bytes in place; readers follow the latest on-chain `manifest_root`. This matches the read/write/commit pattern of traditional filesystems and of runner job execution.

4.2 Storage Deal Markets (Filecoin, Sia)

Filecoin models storage as a bilateral deal between client and miner: the client proposes a duration (180 days to 3 years) and a price; the miner commits and produces periodic Proof-of-Spacetime and Proof-of-Replication proofs. Sia uses a similar file-contract model with ~1 month renewals and a 10/30 erasure code.

Deals are the wrong abstraction for agent workloads:

- **Duration mismatch.** A runner job lasts minutes to hours. A Filecoin deal lasts months. An agent that wanted job-scoped storage on Filecoin would either lock up capital for irrelevant timescales or constantly renegotiate.
- **Per-deal chain overhead.** Every new volume, extension, or modification requires a deal transaction. Cowboy actors create volumes in a single call and let the Relay Registry assign nodes automatically.
- **Retrieval is a separate market.** Filecoin retrieval is asynchronous and can take minutes. Agent workloads need sub-second access.

CBFS/RAS replaces per-client deals with a **single shared Relay Registry**. Relays stake once, serve many volumes, and earn per-byte per-epoch rent with no renegotiation. Runner CapTokens issued at job dispatch give exact access for the job horizon and expire automatically.

4.3 Permaweb (Arweave)

Arweave's **one-shot endowment** model pays upfront for ~200 years of storage, financed by an interest-bearing reserve. Proof-of-Access and the blockweave provide cryptographic guarantees that historical data remains available. This is an elegant solution for truly permanent public data — audit logs, historical records, NFT provenance, immutable publications.

It is the wrong economic model for autonomous agents:

- **Agent data should be deletable.** GDPR, cost control, retractions, and simple obsolescence all require explicit eviction. Arweave offers no mechanism to stop paying.
- **Overwrite cost is linear in writes.** Every update is a fresh endowment. Caches and working state would pay permanent storage prices for data that lives for hours.
- **No access control.** Arweave assumes public data. Agent workloads routinely process private data — user prompts, API keys, personal datasets.

CBFS/RAS uses **per-epoch rent with explicit eviction**: pay for what you use, stop paying and the data goes away. For truly permanent public content, Arweave is better; for Cowboy's workload, it is the wrong economics.

4.4 Client-Encrypted S3 Alternatives (Storj)

Storj is the closest philosophical cousin to CBFS/RAS: client-side encryption, erasure coding (29/80 Reed-Solomon), self-serve buckets, and an S3-compatible API. Key differences:

- **Central Satellite coordination.** Every Storj operation is brokered by a Satellite — a trusted service that holds metadata and issues access grants. CBFS validates CapTokens directly at Relay Nodes against cached chain state, with no central broker on the data path.
- **Erasure overhead.** Storj's 29/80 scheme costs $\sim 2.76\times$ in storage to achieve high durability. CBFS defaults to 4/6 ($1.5\times$) tolerating any 2 simultaneous node failures per object, and owners can opt into higher schemes up to 16/24 when they need more durability.
- **Standalone product.** Storj is not integrated with a smart contract platform. Payments, identity, and access live in its own stack. CBFS/RAS is native to Cowboy: volumes bill from the same account that runs actors, CapTokens integrate with job dispatch, relays register via the same PoS stake primitives, and storage usage flows into the same economic accounting as compute.

4.5 Blob Storage with DA (Walrus, Celestia, EigenDA)

Walrus (on Sui) uses Red-Stuff encoding to provide high-durability blob storage with tight chain integration. Like CBFS, it anchors blobs on-chain and distributes coded fragments across storage nodes. It is immutable and blob-oriented rather than mutable and path-oriented, and it lacks CBFS's CapToken scoping and POSIX surface.

Celestia and EigenDA are **data availability layers**, not storage systems: they guarantee that rollup data is retrievable for a bounded window (weeks), typically via polynomial commitments

and data-availability sampling. They solve a different problem and are not directly comparable — Cowboy uses consensus-level storage anchoring plus CBFS for durable file storage, not a DA layer.

4.6 Tabular Summary

System	Mutation model	Encryption	Access control	Chain writes per read/write	Storage overhead	Target use case
CBFS / RAS	Mutable path-addressed, Merkle root	Client-side AES-256-GCM (PRI-VATE)	CapTokens (owner-signed and runner chain-issued)	0 on data path	$1.5\times$ (4/6 default)	Agent working state, runner attachments, mutable private storage
IPFS	Immutable CID; IPNS for pointers	None (optional client-side)	None (knowledge of CID)	0 (no economic layer)	Pinning-dependent	Public content distribution
Filecoin	Immutable per deal	Optional client-side	Per-deal	Per deal + per retrieval	$\sim 2\times$	Long-term archival
Arweave	Immutable, permanent	Optional client-side	None (public)	Per write (endowment)	Full replication	Permanent public storage
Storj	Mutable object-addressed	Client-side	Macaroon access grants via Satellite	Off-chain (not crypto-native)	$\sim 2.76\times$ (29/80)	S3-compatible decentralized storage
Sia	Mutable via file contracts	Client-side	Per-contract	Per contract + renewal	$3\times$ (10/30)	Low-cost cloud alternative

System	Mutation model	Encryption	Access control	Chain writes per read/write	Storage overhead	Target use case
Swarm	Immutable chunks	None (optional client-side)	None (knowledge of chunk)	Per postage stamp	Optional EC	Ethereum-native web-scale content
Walrus	Immutable blob	Optional client-side	Per-blob object ownership	Per blob registration	Red-Stuff (~4–5×)	High-durability blob storage on Sui

4.7 Why CBFS/RAS for Cowboy

Four properties are jointly load-bearing for Cowboy’s workload and jointly absent from every alternative above:

1. **Off-chain data plane with zero per-access chain overhead.** Reads and writes travel as QUIC RPCs directly between runner and Relay Node, authenticated by a CapToken and verified locally against cached chain state. The only on-chain event on the data path is the periodic manifest root commit, at a cadence chosen by the actor. IPFS has no economic layer; Filecoin and Sia tie every significant action to a deal or contract; Arweave pays upfront but reads via gateways. CBFS gives agents the latency of a private storage system and the durability of a staked public one.
2. **Runner-scoped capability tokens.** Cowboy’s Storage Manager mints CapTokens with exact scope — volume, path prefix, mode, byte quota, TTL — bound to a specific runner address at job dispatch. Swarms of ephemeral agents can share a volume with non-overlapping write prefixes and automatic expiry. No other decentralized storage system models per-job access this way; all assume account-level ownership and require the owner to be online to re-sign every operation.
3. **Mutable path-addressed storage with deterministic consistency.** Working state, artifact caches, and runner intermediate results are fundamentally mutable. CBFS’s manifest-and-root model provides mutability with READ_COMMITTED consistency via the on-chain `manifest_root`, without forcing the re-writing and indirection overhead of content-addressed systems.

4. **POSIX-native FUSE mount as a first-class surface.** Modern LLM agents have filesystem tools — `Read`, `Write`, `Bash`, `grep`, `jq`, `find` — as first-class primitives in their operating vocabulary. A FUSE mount at `/mnt/workspace` lets an agent use those tools directly against persistent storage with no bespoke storage SDK. CBFS is the only decentralized storage system in this comparison that treats a POSIX mount as primary rather than as an afterthought.

For use cases outside this envelope — permanent public content, long-term archival at the lowest possible cost, pure content addressing for deduplication — one of the systems above will serve better. Cowboy does not attempt to replace them; it offers the primitive that they do not provide, the one its autonomous agents actually need.

5 Volumes and the Account Model

An account MAY own up to `MAX_VOLUMES_PER_ACCOUNT` volumes (default **256**). Each volume is identified by:

```
volume_id = keccak256(owner_address || volume_name)
```

`volume_name` MUST be UTF-8, $\leq$ **64 bytes**, and match `[a-zA-Z0-9_\-.]`. Each volume has a fixed **visibility** set at creation and never changed:

- **PRIVATE:** Objects are AES-256-GCM encrypted under a per-volume DEK. The manifest is encrypted. Reads and writes require CapTokens.
- **PUBLIC:** Objects are plaintext. The manifest is plaintext. Reads are open to any party without a CapToken; writes still require one.

An account MAY hold up to `MAX_OBJECTS_PER_VOLUME = 1,000,000` objects per volume, each $\leq$ `MAX_OBJECT_SIZE = 1 GiB`, with a total per-volume ceiling of `MAX_VOLUME_SIZE = 100 GiB` in v1. Object paths are UTF-8 strings of up to `MAX_OBJECT_PATH_LENGTH = 512 bytes`. The full addressing scheme for an object is:

```
/owner_address/volume_name/object_path
```

6 The CBFS Data Plane

CBFS is the substrate that stores and serves volume data. It is implemented as a Rust workspace under `cbfs/` with the following subcrates:

Crate	Responsibility
<code>cbfs-types</code>	Shared protocol types: <code>VolumeId</code> , <code>ShardId</code> , <code>NodeId</code> , <code>PlacementRecord</code> , <code>ObjectDescriptor</code> , <code>RpcRequest/Response</code> , <code>Operation</code> .
<code>cbfs-crypto</code>	AES-256-GCM, BLAKE3, DEK generation and envelope wrapping.
<code>cbfs-erasure</code>	Reed-Solomon over $GF(2^8)$ via <code>reed-solomon-erasure</code> v6 with SIMD.
<code>cbfs-transport</code>	QUIC client/server (<code>quinn</code>) with TLS 1.3 and bincode-framed RPCs.
<code>cbfs-store</code>	Sled-backed blob store on Relay Nodes.
<code>cbfs-manifest</code>	Client-side volume index: sorted <code>BTreeMap<String, ManifestEntry></code> with a Merkle root.
<code>cbfs-placement</code>	Persistent <code>PlacementStore</code> for shard-to-node assignments with CAS and replica merge.
<code>cbfs-hooks</code>	Pluggable traits (<code>AuthProvider</code> , <code>ManifestRegistry</code> , <code>NodeSelector</code> , <code>MeteringSink</code>) with standalone and Cowboy-mode implementations.
<code>cbfs-auth</code>	Cowboy-facing auth runtime: delegation loading, request signing, owner token minting, relay and chain-state caches.
<code>cbfs-sdk</code>	High-level Volume API: create, open, put, get, commit, with local manifest and placement synchronization.
<code>cbfs-fuse</code>	POSIX FUSE mount (inode table, write-back cache, sync daemon, extended attributes).
<code>cbfs-node</code>	The Relay Node daemon: QUIC server, garbage collection, heartbeats, repair, drain.

`cbfs-cli` Operator CLI: `login`, `auth`, `volume`, `put`, `get`,
`mount`, `commit`, `rebalance`.

6.1 Object Representation

Each object is described by an **ObjectDescriptor** stored in the manifest:

Field	Type	Meaning
<code>object_path</code>	<code>String</code>	≤ 512 bytes, UTF-8
<code>write_id</code>	<code>[u8; 16]</code>	Fresh CSPRNG per write, isolates versions
<code>content_hash</code>	<code>[u8; 32]</code>	BLAKE3 of plaintext
<code>ciphertext_hash</code>	<code>[u8; 32]</code>	BLAKE3 of ciphertext before erasure coding
<code>encryption_nonce</code>	<code>[u8; 12]</code>	Fresh random per write (AES-GCM invariant)
<code>size_bytes</code>	<code>u64</code>	Plaintext size
<code>ciphertext_size</code>	<code>u64</code>	Pre-padding ciphertext size (required for erasure truncation)
<code>shard_id</code>	<code>[u8; 32]</code>	<code>BLAKE3(volume_id object_path write_id)</code> — opaque to Relay Nodes
<code>erasure_k</code> , <code>erasure_m</code>	<code>u8</code> , <code>u8</code>	Per-object erasure parameters
<code>shard_hashes</code>	<code>[[u8; 32]; K+M]</code>	BLAKE3 of each shard, used for integrity and PoR

The opacity of `shard_id` — Relay Nodes see a hash, not a path — preserves privacy: they cannot learn which object a shard belongs to, nor detect overwrites. This property forces a specific trust decomposition in the CapToken model described in §2.

6.2 Placement

Shards are assigned to K+M distinct Relay Nodes via a **PlacementRecord**:

Field	Type	Meaning
<code>shard_id</code>	<code>[u8; 32]</code>	Opaque shard identifier
<code>version</code>	<code>u64</code>	CAS version for atomic updates during repair
<code>assignments</code>	<code>[{shard_index: u8, node_id: [u8;32]}]</code>	Node per shard index
<code>erasure_k, erasure_m</code>	<code>u8, u8</code>	Duplicated from descriptor for repair without manifest
<code>ciphertext_size</code>	<code>u64</code>	Duplicated to enable reconstruction without decryption
<code>shard_hashes</code>	<code>[[u8; 32]; K+M]</code>	Duplicated for integrity checks during repair

PlacementRecords are replicated across all assigned Relay Nodes via three RPCs:

- `PutPlacement(shard_id, record, expected_version)` — CAS store/update.
- `GetPlacement(shard_id)` — fetch.
- `ReplicatePlacement(shard_id, record)` — peer-to-peer replication, signed by the sending relay's identity key.

This externalization of placement — as opposed to embedding assignments in the (encrypted) manifest — is what allows repair workers to operate on private volumes without decryption rights.

Node selection is pluggable via the `NodeSelector` trait. The default is `pick_least_used()` (select relays with the fewest shards held); the v2 selector extends this to diversity, health, and latency scoring.

6.3 Erasure Coding

Cowboy uses Reed-Solomon over $GF(2^8)$ via the `reed-solomon-erasure` v6 crate with the `simd-accel` feature. Genesis defaults:

Parameter	Default	Bounds
K (data shards)	4	$2 \leq K \leq 16$
M (parity shards)	2	$1 \leq M \leq 8$
Storage overhead	1.5×	—

For the default 4/6 scheme, an object is split into 6 shards distributed across 6 Relay Nodes; any 4 reconstruct the ciphertext, so the system tolerates any 2 simultaneous Relay Node failures per object. Accounts billed for `effective_size` = `raw_size` × (K+M)/K.

Accounts MAY opt into higher redundancy at creation time, subject to `MAX_ERASURE_K` = 16 and `MAX_ERASURE_M` = 8. The conservative 4/6 default is chosen for v1 to minimize storage overhead while the Relay Node network is small; governance may raise defaults as the network matures.

6.4 Transport

Relay Nodes speak QUIC over TLS 1.3 on a single stream per RPC. Each frame is a `[version: u8][length: u32][bincode payload]` tuple capped at `MAX_SHARD_TRANSFER_SIZE` + 64 KiB (default ~1 MiB). The RPC protocol is:

```
{ request_id: [u8;16], op: Operation, shard_id: [u8;32], shard_index: u8,
  auth_token: Vec<u8>, payload: Vec<u8> }
```

`Operation` enumerates `PutShard`, `GetShard`, `DeleteShard`, `GetPlacement`, `PutPlacement`, `ReplicatePlacement`, `Ping`, `RelayHandshake`, `SyncPeers`, `PeerRelayPushShard`, `PeerPlacementUpdate`. Responses carry a matching `request_id` and a status code (`Ok`, `ErrAuth`, `ErrNotFound`, `ErrCapacity`, `ErrInvalid`, `ErrConflict`).

6.5 FUSE Mount

CBFS ships a FUSE filesystem (`cbfs-fuse`) that exposes a volume as a POSIX directory with write-back caching. Writes land instantly in an in-memory dirty cache; a background sync daemon encrypts, shards, and pushes to Relay Nodes every `DEFAULT_SYNC_INTERVAL` = 5 seconds ($\geq$ `MIN_SYNC_INTERVAL` = 1 second). The local cache is bounded by `MAX_LOCAL_CACHE_SIZE` = 10 GiB per mount.

Extended attributes under the `cbfs.*` prefix expose shard and encryption metadata read-only:

Attribute	Value
<code>cbfs.status</code>	<code>committed</code> / <code>dirty</code> / <code>syncing</code>
<code>cbfs.encrypted</code>	<code>aes-256-gcm</code>
<code>cbfs.erasure</code>	K+M (e.g., 4+2)
<code>cbfs.shards</code>	<code>shard_index@node_addr,...</code>
<code>cbfs.content_hash</code>	First 8 bytes of BLAKE3, hex

Rationale for FUSE as a first-class surface: modern LLM coding agents (Claude, Kimi, GPT) already have filesystem tools (`Read`, `Write`, `Bash`). A FUSE mount lets runners use those native tools against persistent storage without bespoke storage APIs, and composes naturally with standard Unix tools (`grep`, `jq`, `find`). Volume mounts under Cowboy mode are blocked in v1 pending owner-token refresh (§8.5).

7 The CapToken Authorization Model

Storage authorization is the critical seam between Cowboy’s on-chain governance and CBFS’s off-chain data plane. It is designed against three adversary classes simultaneously:

- **Sessions on-chain** — putting every token in consensus state scales poorly and creates rogue-node minting risk.
- **Transaction per access** — wrapping every read or write in a chain transaction destroys the latency and throughput guarantees a storage system needs.
- **Centralized token broker** — a single online signer is a single point of failure and a censorship vector.

Cowboy resolves this with two complementary token types, both validated entirely at the Relay Node, differentiated by a leading version byte (0 = runner, 1 = owner).

7.1 Runner CapTokens (chain-issued)

A **Runner CapToken** is minted by the Storage Manager (0x0A) at job dispatch and stored on-chain. Its contents:

Field	Type	Meaning
version	u8	0
token_id	[u8; 16]	Deterministically derived
token_secret	bytes	Entropy; validation requires secret match
volume_id	[u8; 32]	Target volume
volume_name	String	For logging, not validation
runner_address	Address	Restricts to a single runner
access_mode	AccessMode	ReadOnly, WriteOnly, ReadWrite
path_prefix	String	Confined subtree (e.g., /workspace/)
max_bytes	u64	Per-token quota
valid_from, valid_until	u64, u64	Block heights
mount, sync_interval	bool, u32	Mount parameters

The `token_id` and `token_secret` are derived deterministically from job metadata, runner address, block height, attachment index, and VRF seed, preventing pre-image attacks. At runtime, a Relay Node validates a presented runner CapToken by looking up the stored copy at `StorageManager[token_id]` and checking byte-for-byte match plus TTL, runner address, volume status, access mode, and path prefix containment.

Runner CapTokens MUST have a validity window of at least `MIN_VOLUME_CAP_TOKEN_BLOCKS = 600,000` blocks (~7 days at 1 s blocks), sufficient for typical job durations.

7.2 Owner CapTokens (client-signed, v1)

An **Owner CapToken** (`OwnerCapTokenV1`) is minted locally by the owner's delegated CBFS key, with no chain involvement:

Field	Type	Meaning
version	u8	1
volume_id	[u8; 32]	Target volume
volume_name	String	For logging, not validation

Field	Type	Meaning
<code>owner_address</code>	Address	Must match <code>delegation_cert.wallet_address</code>
<code>access_mode</code>	AccessMode	ReadOnly, WriteOnly, ReadWrite
<code>path_prefix</code>	String	Confined subtree
<code>valid_from_ms</code> , <code>valid_until_ms</code>	u64, u64	Unix milliseconds (wall-clock, short TTL)
<code>mount</code> , <code>sync_interval</code>	bool, u32	Mount parameters
<code>delegation_cert</code>	DelegationCert	Embedded cert (see §2.3)
<code>owner_signature</code>	bytes	Ed25519 over all other fields, including the version byte

An owner token has a maximum TTL of **5 minutes** for reads and puts, and **4 hours** for mounts (v1 blocks Cowboy-mode mounts pending refresh). Relays never store owner tokens: they are validated purely by signature verification against the embedded delegation cert, the cert against the chain-anchored wallet address, and the wallet against cached chain state (volume ownership, revocation, status).

Canonical signing bytes for an Owner CapToken:

```
payload = bincode(OwnerCapTokenSigningPayload {
    version: 1,
    volume_id, volume_name, owner_address, access_mode, path_prefix,
    valid_from_ms, valid_until_ms, mount, sync_interval, delegation_cert
})
owner_signature = Ed25519Sign(cbfs_private_key, payload)
```

The **version** byte MUST be included in the signature input to prevent version-confusion attacks where an attacker strips or replaces the prefix.

7.3 DelegationCert

A **DelegationCert** binds an Ed25519 CBFS key to an account with a short expiry. It is signed once with the cold wallet key and then reused across many owner tokens:

Field	Type	Meaning
<code>wallet_address</code>	Address	Owner's Cowboy account
<code>cbfs_public_key</code>	[u8; 32]	Delegated Ed25519 key
<code>scope</code>	String	Fixed to "cbfs"
<code>chain_id</code>	u64	Prevents cross-chain replay
<code>network</code>	String	mainnet / testnet / ...
<code>aud</code>	String	Fixed to "cowboy:cbfs"
<code>expires_at_ms</code>	u64	Cert expiry (default 30 days)
<code>signature</code>	bytes	secp256k1 wallet signature

Wallet signing bytes:

```
payload = bincode(DelegationCertSigningPayload {
    wallet_address, cbfs_public_key, scope, chain_id, network, aud, expires_at_ms
})
wallet_signature = Secp256k1Sign(wallet_key, keccak256(payload))
```

The delegated CBFS key is stored encrypted at `~/.cbfs/cbfs_key.enc` under AES-256-GCM with a user-supplied passphrase (or environment variable for headless use), decrypted on demand into `mlock`'d memory, and zeroed after each signing. Rotation and revocation are supported via `cbfs auth rotate` and `cbfs auth revoke`; the latter posts to the Storage Manager's revocation list (§6.1), propagated to Relay Nodes within the cache refresh bound (§5).

7.4 Access Modes

Mode	Read	Write	List	Delete
<code>ReadOnly</code>	Yes	No	Yes	No
<code>WriteOnly</code>	No	Yes	No	No
<code>ReadWrite</code>	Yes	Yes	Yes	No

`Delete` is never granted to runners; it is an owner-only control-plane operation.

7.5 Scoping and Concurrency

CapTokens are scoped on four axes simultaneously: **volume**, **path prefix**, **byte quota**, and **time window**. For write tokens, the Storage Manager MUST enforce non-overlapping path prefixes at issuance time (Layer 1). For read tokens, overlap is permitted. Concurrent write tokens with disjoint prefixes permit swarm-style workloads where multiple runners extend a single volume without coordinating.

Three layers of prefix enforcement apply:

1. **Issuance time (strong)**. Storage Manager rejects new write tokens whose prefix overlaps an existing write token on the same volume.
2. **Commit time (detection)**. A coordinator token holder reads the committed manifest after sub-agent commits and verifies all paths fall within each delegated prefix. Rogue commits are detected and revoked, then corrected.
3. **Write time (weak)**. Relay Nodes cannot verify prefix compliance: shard IDs are opaque hashes by design, so per-shard path-check is impossible. Relay Nodes verify signature, mode, and byte quota; they do not verify that a write targets a permitted path.

The Layer-3 weakness is deliberate: exposing paths to Relay Nodes would break private-volume confidentiality. Instead, a rogue runner with a scoped write token can waste shards out-of-prefix, bounded by its `max_bytes` quota; uncommitted shards are garbage-collected after `ORPHAN_SHARD_TTL = 7,200 blocks (~24 h)`.

8 Volume Lifecycle

This section specifies the end-to-end lifecycle of a volume, distinguishing control-plane (on-chain) and data-plane (CBFS) steps.

8.1 Create (control plane)

The owner submits a `create_volume` transaction to the Storage Manager (0x0A) with `volume_name`, `erasure_k`, `erasure_m`, `visibility`, and an initial CBY reserve. The actor:

1. Derives `volume_id = keccak256(owner_address || volume_name)`. If the volume already exists, revert with `VolumeAlreadyExists`.
2. Charges `VOLUME_CREATION_FEE = 1,000 CBY` from the owner's account.
3. For `PRIVATE` volumes: generates a random 32-byte DEK, envelope-encrypts it under a wrapping key derived `HKDF-SHA256(account_secret, "cbfs-volume-" || volume_id)`, and records `wrapped_dek` plus `wrapping_key_hash` on-chain.

4. Writes a `StorageCommitment` record with status `ACTIVE`, empty `manifest_root`, `effective_size_bytes = 0`.
5. Assigns initial Relay Nodes via the Relay Registry (§7).

8.2 Attach (control plane)

When a job is dispatched with volume attachments, the Storage Manager calls `issue_attachments()` to mint a runner `CapToken` per attachment and (for private volumes) re-encrypts the plaintext DEK to the runner's ephemeral X25519 public key using AES-256-GCM ECIES (`sealed_runner_keys` in `VolumeAttachment`). The runner decrypts the sealed DEK with its ephemeral private key on job start and zeroes it on job completion. This transient plaintext exposure at the Dispatcher is equivalent to the trust already placed in the Dispatcher for job assignment; v2 will route via the Secrets Manager system actor to eliminate it.

8.3 Write (data plane, no on-chain traffic)

The runner (or owner):

1. Generates a fresh 16-byte `write_id` and computes `shard_id = BLAKE3(volume_id || object_path || write_id)`.
2. Encrypts the object: `ciphertext = AES-256-GCM(DEK, nonce = CSPRNG(12), plaintext, aad = canonical_metadata)`.
3. Reed-Solomon encodes the ciphertext to $K+M$ shards of `ceil(ciphertext_size / K)` bytes each, with padding.
4. Selects $K+M$ Relay Nodes from the cached Relay Registry and issues `PutShard(shard_id, shard_index, shard_bytes, metadata)` RPCs in parallel, each authenticated by the `CapToken`.
5. Records the new `ObjectDescriptor` in the local manifest and constructs the `PlacementRecord`.

Every step is local or off-chain. No transaction is submitted.

8.4 Read (data plane, no on-chain traffic)

1. Fetch the manifest root from the authoritative Storage Manager state (or a cached, freshness-bounded value).
2. Fetch the manifest blob from Relay Nodes via `GetShard` on `BLAKE3(volume_id || "__manifest__")`; for private volumes, decrypt with the DEK.

3. Resolve `object_path` to an `ObjectDescriptor`; fetch its `PlacementRecord` via `GetPlacement`.
4. Request any `K` of `K+M` shards in parallel (prefer low-latency, fail over on timeout or corruption).
5. Reconstruct the ciphertext via Reed-Solomon, truncate to `ciphertext_size`, verify `ciphertext_hash`.
6. Decrypt to plaintext, verify `content_hash`.

8.5 Commit (two-level)

Manifest commit is atomic at two levels:

1. **CBFS commit** (`commit()`): Publish the (encrypted) manifest as a well-known shard (`BLAKE3(volume_id || "__manifest__")`) and the `PlacementRecord` for each changed shard to its assigned Relay Nodes via `PutPlacement`. Best-effort; failures are logged but do not block.
2. **Cowboy commit** (`commit_manifest(manifest_root: bytes32)`): Submit only the 32-byte Merkle root to the on-chain `StorageCommitment.manifest_root`. This is the sole on-chain write on the data path, and it is triggered by the owner or runner at a natural boundary (job end, user command), not per object.

This split gives CBFS a `READ_COMMITTED` consistency model across runners: the authoritative manifest is whatever root the chain last accepted.

8.6 Delete, Grace, Restore

The owner MAY soft-delete a volume via `delete_volume`. The `StorageCommitment` transitions to `DELETED` for `VOLUME_DELETE_GRACE_EPOCHS = 7,200 blocks (~24 h)`, during which:

- All active `CapTokens` are revoked.
- New `CapTokens` cannot be issued.
- Storage rent continues to accrue.
- The owner MAY call `undelete_volume` to restore status to `ACTIVE`.

If the grace window expires without restoration, the volume transitions to `GARBAGE_COLLECTING`. Relay Nodes learn of the transition via heartbeat acknowledgments or direct RPCs, purge their shards asynchronously, and storage rent ceases. The same terminal state is reached when per-epoch billing detects a balance shortfall (§8).

9 Relay Nodes

A Relay Node is a dedicated storage participant distinct from validators and runners. It persists erasure-coded shards, serves reads and writes to authorized clients, participates in Relay-to-Relay repair, answers Proof-of-Retrievability challenges, and reports capacity and health to the chain.

9.1 Registration and Stake

A Relay Node registers by submitting a `register_relay(capacity_bytes)` transaction to the Relay Registry (0x0B) with a stake of at least `MIN_RELAY_STAKE CBY` (governance-tunable) and publishing its `relay_identity_pubkey`. The registration produces a **RelayNodeProfile**:

Field	Type	Meaning
<code>address</code>	<code>[u8; 32]</code>	Relay Node address
<code>stake_amount</code>	<code>u256</code>	Self-bonded CBY
<code>capacity_bytes</code>	<code>u64</code>	Advertised capacity
<code>used_bytes</code>	<code>u64</code>	Current shard storage
<code>last_heartbeat</code>	<code>u64</code>	Block height
<code>health</code>	<code>u8</code>	Decays 1/block, max 100, reset on heartbeat
<code>shards_held</code>	<code>u32</code>	Active shard count
<code>shards_lost</code>	<code>u32</code>	Historical losses (reputation)
<code>region_hint</code>	<code>[u8; 4]</code>	Geographic hint for placement diversity

Relay Nodes **MUST** heartbeat via `heartbeat()` at least every `MAX_RELAY_HEALTH = 100 blocks` to remain assignable. Nodes with `health < MIN_HEALTH_FOR_ASSIGNMENT = 50` receive no new assignments; a node that reaches `health = 0` is removed from the active set and its shards are flagged for repair.

A Relay Node **MAY** unstake after a `RELAY_UNSTAKE_DELAY = 7,200 block (~24 h)` cooldown, provided all its shards have been drained (§7.3).

9.2 Repair (autonomous, two-phase)

Relay Nodes run a continuous repair loop, invoked every `REPAIR_CHECK_INTERVAL = 300 blocks`, in two phases:

Phase 1 — self-heal. For each PlacementRecord where this node is an assignee, verify the local shard against its expected `shard_hash`. On missing or corrupt:

1. Fetch K healthy shards from peers listed in the PlacementRecord.
2. Reconstruct the ciphertext via Reed-Solomon, truncating to `ciphertext_size` (this step is why `ciphertext_size` is duplicated into the PlacementRecord — without it, padding would corrupt the reconstruction).
3. Re-encode, extract the expected shard, verify its hash, persist locally.

Self-heal is safe to run concurrently on any number of nodes: it only writes to local storage, never to the PlacementRecord.

Phase 2 — dead-node replacement. For each PlacementRecord, probe all assignees. If a node is unreachable past a timeout:

1. **Leader election.** Among the live assignees, the node with the **lowest** `shard_index` is the unique leader for this PlacementRecord. Ties cannot occur (indices are distinct).
2. The leader selects a replacement Relay Node from the peer list (excluding current assignees) using the `NodeSelector`.
3. The leader reconstructs the missing shard(s) from surviving shards and pushes them to the replacement via `PutShard` or the signed `PeerRelayPushShard` RPC.
4. The leader writes a new PlacementRecord with the updated assignment and `version + 1` via a CAS `PutPlacement(shard_id, record, expected_version)`. If CAS fails (because another node replaced a different dead node in the same record concurrently), the leader retries in the next cycle.
5. Updated PlacementRecord is replicated to all current assignees via `ReplicatePlacement`.

Under the default $K=4, M=2$ scheme, two simultaneous failures per object are tolerated; three before repair completes will cause data loss. The `REPAIR_CHECK_INTERVAL` is chosen aggressively to keep the window tight.

9.3 Drain and Peer-Push

A relay may voluntarily exit (or be commanded to exit by governance) by transitioning to `Draining` status. A drain worker iterates assigned shards and peer-pushes each via the signed RPCs:

- `PeerRelayPushShard(sender_node_id, timestamp, shard_id, shard_index, old_record_hash, shard_bytes_hash, shard_bytes, signature)`
- `PeerPlacementUpdate(sender_node_id, timestamp, old_record_hash, new_record_hash, new_record, signature)`

Both are signed by the sending relay's identity key over domain-tagged BLAKE3 digests (`cbfs:peer_push_shard:v1`, `cbfs:peer_placement_update:v1`) and reject messages whose timestamp is outside a 60 s window. On successful drain, the node writes a signed `RelaySelfDisableRequest` when `shards_remaining == 0` and exits the active set.

Full drain bonds, proof-of-drain sampling, and concurrent-drain coordination are part of Phase 2 (§12).

9.4 Incentives

Relay Nodes earn from two streams:

1. **Per-epoch storage rent.** The total per-epoch rent on a volume is distributed pro rata across the Relay Nodes holding its shards:

```
total_storage_fee = effective_size × STORAGE_FEE_PER_BYTE_PER_EPOCH
per_relay_share   = total_storage_fee / num_relay_nodes_holding_shards
```

2. **Per-byte transfer fees.** Each serving relay receives `bytes_served × TRANSFER_FEE_PER_BYTE` from the reader's account at read time. The exact rates are governance-tunable.

`STORAGE_FEE_BURN_RATE` = 10% of gross storage rent is burned; the remaining 90% flows to relays. Relays with high `shards_lost` values receive fewer assignments over time via the `NodeSelector` reputation score.

10 Proof of Retrievability

Because Relay Nodes are economically incentivized to silently discard shards (to claim rent without serving), RAS implements periodic **Proof of Retrievability (PoR)** challenges. Challenges are cheap to verify, expensive to fake, and cost Relay Nodes measurable stake when they fail.

10.1 Challenge Protocol

A challenge is triggered by an on-chain timer every `POR_CHALLENGE_INTERVAL` = 600 blocks (~2 h). It selects a random (`shard_id`, `shard_index`, `byte_offset`, `byte_length`) tuple targeting an active relay. The relay **MUST** respond within `POR_RESPONSE_WINDOW` = 50 blocks (~10 min) with:

- The requested byte range from the named shard.
- A two-level Merkle proof chain:

1. `byte_range_proof` — links the byte range to the shard's `shard_hash`.
2. `shard_inclusion_proof` — links `shard_hash` to the on-chain `manifest_root` of the target `StorageCommitment`.

The on-chain verifier:

1. Checks `BLAKE3(byte_range_data)` equals the leaf of `byte_range_proof`.
2. Verifies `byte_range_proof` resolves to `shard_hash`.
3. Verifies `shard_inclusion_proof` resolves to `manifest_root`.

A relay holding the shard answers trivially; one that discarded it cannot. Manifest updates during the response window are handled by evaluating proofs against the `manifest_root` active at challenge issuance; shards removed by a legitimate write during the window void the challenge without penalty.

10.2 Outcomes

Outcome	Consequence
Valid response within window	No action
No response	<code>shards_lost++</code> , shard flagged for repair, <code>POR_MISS_PENALTY</code> slashed
Invalid proof	<code>shards_lost++</code> , shard flagged for repair, <code>POR_FRAUD_PENALTY</code> (> miss) slashed
3+ consecutive misses	Removed from active set, all shards flagged for repair, <code>RELAY_EVICTION_PENALTY</code> slashed

A governance-tunable `POR_CHALLENGE_FEE_SHARE` = 2% of storage rent funds challenge issuance; challengers (who may be any on-chain actor triggering challenges via timers — see Technical Whitepaper §3.3) receive a finder's fee from slashed stake.

11 Billing, Rent, and Eviction

Storage rent is CBFS's primary economic mechanism: it creates a continuous cost for occupying Relay Node capacity, directly fundable by any account.

11.1 Fee Components

Fee	When Charged	Amount
Volume creation	<code>create_volume</code>	<code>VOLUME_CREATION_FEE = 1,000 CBY</code>
Attachment	<code>submit_task</code> with volume attachment	<code>BASE_ATTACHMENT_FEE = 100 CBY</code> per volume per job
Storage rent	Per epoch	<code>effective_size × STORAGE_FEE_PER_BYTE_PER_EPOCH</code> (governance-tunable)
Transfer	Per read/write byte	<code>bytes × TRANSFER_FEE_PER_BYTE</code> (governance-tunable)

These are separate from the Cycle and Cell fee markets specified in the Technical Whitepaper §4 and §17; RAS does NOT introduce a third on-chain meter. On-chain operations (creating StorageCommitments, writing manifest roots, posting revocations) consume Cycles and Cells normally. The per-epoch storage rent is a separate ledger maintained by the Storage Manager, debited from a dedicated `balance_reserved` on the account.

11.2 Settlement

At each epoch boundary, the Storage Manager runs `settle_volume_epoch()`:

1. Compute `epoch_fee = effective_size_bytes × STORAGE_FEE_PER_BYTE_PER_EPOCH`.
2. If `balance_reserved ≥ epoch_fee`: charge, split 10% burn / 90% to relays (pro rata by shards held), increment `last_billed_epoch`.
3. If `balance_reserved < epoch_fee`: mark the volume as entering `GRACE_PERIOD`, with reads permitted and writes denied.
4. If the volume has been in `GRACE_PERIOD` for `STORAGE_GRACE_EPOCHS = 7,200 blocks` (~24 h) without top-up, transition to `GARBAGE_COLLECTING` and broadcast a purge signal to holding relays.

Transfer fees are settled per read via Relay Node attestations, batched and submitted as a single transaction per relay per epoch to minimize on-chain overhead.

11.3 Comparison with State Rent

The mechanism intentionally mirrors the state rent system specified in Technical Whitepaper §4.4 and §17.5 — grace period, then warning, then eviction — but differs in kind:

Property	State rent (on-chain state)	Storage rent (CBFS volumes)
Target resource	QMDB key/value state	Relay Node shard capacity
Billed unit	Cells (bytes of consensus state)	Bytes of <code>effective_size</code> (post-erasure)
Grace period	7 rent-epochs	7,200 blocks (~1 rent-epoch at 1 day)
Eviction	Storage pruned, code + root preserved	Shards purged, <code>StorageCommitment</code> retained until GC completes
Restoration	Anyone may restore with original data + back-rent	Owner may <code>undelete_volume</code> during grace; not after GC

12 On-Chain Control Plane

Two system actors anchor RAS on Cowboy, both specified in the Technical Whitepaper §9 and detailed below. Their storage keys follow the pattern specified in CIP-4.

12.1 Storage Manager (0x0A)

The Storage Manager is the on-chain authority for volume ownership, CapToken issuance, revocation, and billing. Its state comprises:

Key	Value
<code>ras:volume:{volume_id}</code>	<code>StorageCommitment</code> (owner, manifest root, size, erasure K/M, relays, status)
<code>ras:owner-name:{owner_address}:{keccak256(volume_name)}</code>	<code>volume_id</code> (name→id index)

Key	Value
<code>ras:token:{token_id}</code>	<code>CapToken</code> (runner tokens, stored for validation)
<code>ras:relay-assignments:{volume_id}</code>	List of assigned relay addresses
<code>ras:account-summary:{owner_address}</code>	<code>AccountStorageSummary</code> (total volumes, effective bytes, reserved balance)
<code>ras:volume-billing:{volume_id}</code>	<code>VolumeBillingState</code> (last_billed_epoch, delete_requested_epoch, accrued_fees)
<code>ras:challenge:{challenge_id}:{wallet_address}</code>	<code>ChallengeRecord</code> (single-use nonce for high-impact ops, 30 s TTL)
<code>ras:revocation:{wallet_address}:{cbfs_public_key}</code>	<code>DelegationRevocation</code> (timestamp)

The `StorageCommitment` (`types.rs`) holds:

```

volume_id:      bytes32      // keccak256(owner || volume_name)
owner:          Address
volume_name:    String       // UTF-8, ≤64 bytes
visibility:     PRIVATE | PUBLIC
created_at:     u64          // block height
wrapped_dek:    bytes        // envelope-encrypted volume DEK (empty for PUBLIC)
wrapping_key_hash: bytes32
manifest_root:  bytes32      // Merkle root of manifest
raw_size_bytes: u64
effective_size_bytes: u64      // raw × (K+M)/K
erasure_k:      u8
erasure_m:      u8
last_updated:   u64          // block height of last manifest commit
degraded_shards: u16
status:         ACTIVE | GRACE_PERIOD | DELETED | GARBAGE_COLLECTING

```

The Storage Manager exposes `/ras/*` RPC endpoints on Cowboy validator nodes, including:

Endpoint	Purpose
POST /ras/challenge	Mint a single-use challenge (TTL 30 s) for high-impact mutations
GET /ras/auth/me	Verify delegation, return account summary
POST /ras/auth/revoke	Revoke a DelegationCert
GET /ras/accounts/balance	Check <code>balance_reserved</code> and current usage
POST /ras/volumes	Register a new volume (requires challenge)
GET /ras/volumes	List owned volumes
GET /ras/volumes/{id}	Volume details, relay endpoints, status
GET /ras/volumes/{name}/open	Metadata sufficient to construct an Owner CapToken
POST /ras/volumes/{id}/delete	Soft-delete (enter grace)
POST /ras/volumes/{id}/undelete	Cancel soft-delete within grace

All mutations require a delegation envelope: `X-Cbfs-Delegation` (base64url DelegationCert), `X-Cbfs-Timestamp`, `X-Cbfs-Nonce` or `X-Cbfs-Challenge-Id`, and `X-Cbfs-Signature` (Ed25519 over the canonical request payload). Timestamps MUST be within ± 30 s; nonces and challenge IDs are tracked to prevent replay.

12.2 Relay Registry (0x0B)

The Relay Registry manages Relay Node membership, health, and settlement:

Key	Value
<code>ras:relay:{relay_address}</code>	<code>RelayNodeProfile</code> (by address)
<code>ras:relay-node:{node_id}</code>	<code>RelayNodeProfile</code> (by node_id)
<code>ras:active-relays</code>	List of active relay addresses
<code>ras:settlement:{relay_address}</code>	<code>RelaySettlementState</code> (claimable rewards)

Relay Nodes perform `heartbeat()`, `advertise_capacity()`, `report_shards()`, and `claim_settlement()` against this actor. It is also the target of PoR challenge resolution and slashing.

12.3 Cache Freshness

Relay Nodes maintain local caches of Storage Manager and Relay Registry state — ownership, revocations, relay profiles — to avoid a chain round-trip on every CapToken validation. Caches refresh every **15 seconds** with a maximum staleness bound of **60 seconds**. A revoked DelegationCert is therefore effective across the network within 60 s of the on-chain posting. Relays missing from a local cache are fetched synchronously on first use, preventing a latency cliff for newly registered relays.

13 State Transition Function (Storage Subset)

This section specifies the storage-relevant portion of the state transition function defined in Technical Whitepaper §State Transition Function. Within each block:

1. **Process RAS system instructions.** Volume creation, deletion, undelete, commit manifest, revoke delegation, and settlement apply to Storage Manager state; relay registration, heartbeat, drain, and settlement apply to Relay Registry state.
2. **Issue runner CapTokens.** For each runner job assigned in the block with volume attachments, derive and persist the runner CapToken and sealed DEK copies.
3. **Resolve PoR challenges.** For challenges whose response window closes this block, verify submitted proofs and apply slashing and reputation updates.
4. **Epoch boundary (every `rent_epoch_length`).** Iterate all active volumes, compute storage rent, transition statuses (`ACTIVE` → `GRACE_PERIOD` → `GARBAGE_COLLECTING`, with restoration to `ACTIVE` permitted from `GRACE_PERIOD`), and distribute relay earnings.

All storage-related mutations are subject to the normal Cycle/Cell fee markets; the per-epoch rent lives in its own accounting ledger as described in §8.

14 Normative Conventions

This document uses **MUST/SHOULD/MAY** as defined in RFC 2119. Parameters marked **governance-tunable** can be changed via the on-chain governance mechanism specified in Technical Whitepaper §11. In all cases, the Technical Whitepaper is the normative reference for consensus, execution, and economic parameters outside the storage layer.

15 1. Volume Model

15.1 1.1. Volume identity

`volume_id = keccak256(owner_address || utf8_bytes(volume_name))`

`volume_name` MUST be UTF-8, NFC-normalized, of length 1–64 bytes, matching `^[a-zA-Z0-9_\-\.]+\.`. It MUST NOT begin or end with `.` or `-`.

15.2 1.2. Visibility

`visibility` $\in \{ \text{PRIVATE}, \text{PUBLIC} \}$. MUST be set at creation and MUST NOT change thereafter. PRIVATE volumes' objects and manifests MUST be encrypted under a per-volume DEK; PUBLIC volumes MUST NOT encrypt objects or manifests.

15.3 1.3. Object identity and descriptor

Each object MUST carry a fresh `write_id` per write, a content hash over plaintext, a ciphertext hash over the encrypted form (for PRIVATE volumes), and the size and padding metadata required to reconstruct without decryption. Descriptor fields are enumerated in §4.

15.4 1.4. Quotas

An account MUST NOT own more than `MAX_VOLUMES_PER_ACCOUNT` volumes. A volume MUST NOT contain more than `MAX_OBJECTS_PER_VOLUME` objects. An object MUST NOT exceed `MAX_OBJECT_SIZE`. A volume MUST NOT exceed `MAX_VOLUME_SIZE`.

16 2. CapTokens

16.1 2.1. Encoding

CapTokens MUST be encoded as `[version: u8][bincode payload]` where `version = 0` (runner) or `version = 1` (owner). The version byte MUST be included in all signing inputs that cover the token.

16.2 2.2. Runner CapTokens

A runner CapToken MUST be produced by the Storage Manager at job dispatch, MUST be stored on-chain under `StorageManager[token_id]`, and MUST include `token_id`, `token_secret`, `volume_id`, `runner_address`, `access_mode`, `path_prefix`, `max_bytes`, and `[valid_from, valid_until]` block heights. Relay Nodes MUST validate a presented runner CapToken by byte-for-byte comparison against the stored copy plus TTL, runner address, access mode, and path prefix containment.

16.3 2.3. Owner CapTokens (v1)

An Owner CapToken MUST carry an embedded `DelegationCert` and an `owner_signature` computed via Ed25519 over the canonical `OwnerCapTokenSigningPayload` including the `version` byte. Relay Nodes MUST verify, in order: the wallet secp256k1 signature on the `DelegationCert`, the `owner_signature` against `delegation_cert.cbfs_public_key`, the cert expiry, `chain_id`, `network`, `aud == "cowboy:cbfs"`, `scope == "cbfs"`, the cert not being present in the Storage Manager's revocation list, the volume's `status`, the token's TTL (with a ± 60 s clock-skew tolerance), `access_mode` authorizing the requested operation, `path_prefix` containing the requested path, and `owner_address == delegation_cert.wallet_address`.

16.4 2.4. TTL caps

Owner CapToken TTL MUST NOT exceed **5 minutes** for non-mount operations and **4 hours** for mount operations. Runner CapToken TTL MUST be at least `MIN_VOLUME_CAP_TOKEN_BLOCKS = 600,000` blocks.

16.5 2.5. Delegation certs

A `DelegationCert` MUST be signed by the wallet secp256k1 key over `keccak256(bincode(DelegationCertSigningPayload))`. It MUST expire no more than **30 days** after issuance.

16.6 2.6. Prefix overlap (writes)

The Storage Manager MUST reject a runner CapToken issuance with `access_mode ∈ {WriteOnly, ReadWrite}` whose `path_prefix` overlaps the `path_prefix` of any already-valid write token on the same volume.

17 3. Encryption

17.1 3.1. DEK generation

A PRIVATE volume's DEK MUST be 32 bytes drawn from a CSPRNG. It MUST be envelope-encrypted under a wrapping key derived as `HKDF-SHA256(account_secret, "cbfs-volume-" || volume_id)` before on-chain storage.

17.2 3.2. Object encryption

Each object MUST be encrypted with AES-256-GCM using a freshly drawn 12-byte nonce per write. Deterministic nonces MUST NOT be used. The AAD MUST cover canonical metadata that binds the ciphertext to its descriptor (`volume_id`, `object_path`, `write_id`). The resulting tag MUST be appended to the ciphertext per RFC 5116.

17.3 3.3. Content and ciphertext hashes

`content_hash = BLAKE3(plaintext)` MUST be recorded. `ciphertext_hash = BLAKE3(ciphertext)` MUST be recorded and verified by readers prior to decryption.

17.4 3.4. Nonce uniqueness

A given (DEK, nonce) pair MUST NOT be reused across writes. The fresh `write_id` ensures `shard_id` differs between overwrites; the independent random nonce ensures GCM safety even if `write_id` is reused by accident.

18 4. Manifest and Placement

18.1 4.1. Manifest

A manifest MUST be a sorted `BTreeMap<String, ManifestEntry>` with variants for files, symlinks, and directories. Its Merkle root MUST be computed deterministically and committed on-chain as `StorageCommitment.manifest_root`.

18.2 4.2. Manifest shard address

A manifest's canonical shard ID MUST be `BLAKE3(volume_id || "__manifest__")`. The manifest MUST be stored as K+M erasure-coded shards at this address.

18.3 4.3. PlacementRecord

PlacementRecords MUST be mutable, versioned records distinct from the manifest, replicated across all assigned Relay Nodes. Updates MUST use compare-and-swap (`PutPlacement(shard_id, record, expected_version)`).

18.4 4.4. PlacementRecord fields

A PlacementRecord MUST duplicate `erasure_k`, `erasure_m`, `ciphertext_size`, and `shard_hashes` from the ObjectDescriptor so that repair workers can reconstruct without decrypting the manifest.

19 5. Erasure Coding

19.1 5.1. Scheme

Reed-Solomon over $GF(2^8)$ MUST be used. The `reed-solomon-erasure v6` crate with `simd-accel` is the reference implementation.

19.2 5.2. Parameters

Genesis defaults: $K=4$, $M=2$. Implementations MUST support $2 \leq K \leq \text{MAX_ERASURE_K} = 16$ and $1 \leq M \leq \text{MAX_ERASURE_M} = 8$.

19.3 5.3. Padding

Ciphertext MUST be padded to a multiple of K before splitting. `ciphertext_size` MUST be stored so readers can truncate after reconstruction.

19.4 5.4. Integrity

Each shard MUST be hashed with BLAKE3; the hash MUST be stored in both the ObjectDescriptor and PlacementRecord. Readers MUST verify shard integrity before reconstruction; repair workers MUST verify reconstructed shards before persistence.

20 6. Relay Nodes

20.1 6.1. Registration

A Relay Node MUST submit `register_relay(capacity_bytes)` with a self-bond of at least `MIN_RELAY_STAKE` CBY. It MUST publish a `relay_identity_pubkey` (Ed25519) used for peer-relay RPC authentication.

20.2 6.2. Heartbeat and health

`health` MUST decay 1/block, reset to `MAX_RELAY_HEALTH = 100` on each `heartbeat()`. A relay with `health < MIN_HEALTH_FOR_ASSIGNMENT = 50` MUST NOT receive new shard assignments. A relay that reaches `health = 0` MUST be removed from the active set and its shards MUST be flagged for repair.

20.3 6.3. Unstaking

Relay unstaking MUST be gated by a `RELAY_UNSTAKE_DELAY = 7,200` block cooldown after full drain completion.

20.4 6.4. Peer-relay RPC auth

`PeerRelayPushShard` and `PeerPlacementUpdate` MUST be signed by the sending relay's identity key over domain-tagged BLAKE3 digests (`cbfs:peer_push_shard:v1`, `cbfs:peer_placement_update:v1`). Messages MUST be rejected if their timestamp is outside a 60 s window.

20.5 6.5. Repair cadence

Repair scans MUST run at most every `REPAIR_CHECK_INTERVAL = 300` blocks. Orphan shards (shards without a referencing manifest) MUST be garbage-collected after `ORPHAN_SHARD_TTL = 7,200` blocks.

21 7. Proof of Retrievability

21.1 7.1. Challenge cadence

On-chain challenge timers **MUST** fire at intervals of `POR_CHALLENGE_INTERVAL = 600 blocks`. Challenges **SHOULD** be funded from `POR_CHALLENGE_FEE_SHARE = 2%` of storage rent.

21.2 7.2. Response window

A challenged Relay Node **MUST** respond within `POR_RESPONSE_WINDOW = 50 blocks`. Failure to respond **MUST** trigger `POR_MISS_PENALTY` slashing, reputation decrement, and shard repair.

21.3 7.3. Proof structure

A response **MUST** contain the requested byte range and a two-level Merkle chain: a proof from the byte range to `shard_hash`, and a proof from `shard_hash` to the on-chain `manifest_root` at the time of challenge issuance.

21.4 7.4. Slashing

Invalid proofs **MUST** trigger `POR_FRAUD_PENALTY > POR_MISS_PENALTY`. Three consecutive failures **MUST** trigger `RELAY_EVICTION_PENALTY` and removal from the active set.

22 8. Billing and Fees

22.1 8.1. Fee components

Volume creation **MUST** charge `VOLUME_CREATION_FEE`. Attachment **MUST** charge `BASE_ATTACHMENT_FEE` per volume per job. Storage rent **MUST** accrue at `STORAGE_FEE_PER_BYTE_PER_EPOCH` on `effective_size_bytes`. Transfer fees **MUST** be `TRANSFER_FEE_PER_BYTE`, payable to the serving relay.

22.2 8.2. Burn split

`STORAGE_FEE_BURN_RATE = 10%` of gross storage rent **MUST** be burned. The remainder **MUST** be distributed pro rata to holding relays.

22.3 8.3. Grace period

If a volume's `balance_reserved < epoch_fee` at an epoch boundary, the volume MUST transition to `GRACE_PERIOD` with writes denied and reads permitted. A volume in `GRACE_PERIOD` for `STORAGE_GRACE_EPOCHS = 7,200 blocks` without balance restoration MUST transition to `GARBAGE_COLLECTING`.

22.4 8.4. Soft-delete grace

An owner-initiated `delete_volume` MUST place the volume in `DELETED` status for `VOLUME_DELETE_GRACE_EPOCHS = 7,200 blocks`, during which `undelete_volume` MAY restore it. After grace, the volume MUST transition to `GARBAGE_COLLECTING`.

22.5 8.5. No third meter

RAS MUST NOT introduce a third on-chain meter. On-chain operations related to storage MUST consume Cycles and Cells per Technical Whitepaper §4; storage rent MUST be maintained in the Storage Manager's separate billing ledger.

23 9. RPC and Wire Format

23.1 9.1. Transport

CBFS RPCs MUST run over QUIC with TLS 1.3. Connections MUST be authenticated per §6.4 for peer-relay traffic; client-to-relay connections MUST authenticate via CapToken per §2.

23.2 9.2. Frame format

Each RPC frame MUST be encoded as `[version: u8][length: u32 big-endian][bincode payload]`. Frames MUST NOT exceed `MAX_SHARD_TRANSFER_SIZE + 64 KiB`.

23.3 9.3. RPC operations

Implementations MUST support at minimum: `PutShard`, `GetShard`, `DeleteShard`, `GetPlacement`, `PutPlacement`, `ReplicatePlacement`, `Ping`, `RelayHandshake`, `SyncPeers`, `PeerRelayPushShard`, `PeerPlacementUpdate`.

23.4 9.4. Relay handshake

`RelayHandshake` MUST sign `keccak256(node_id || client_nonce || timestamp_ms)` with the relay identity key. Clients MUST verify against the cached on-chain `relay_identity_pubkey`; timestamps outside ± 30 s MUST be rejected.

24 10. Security Considerations

24.1 10.1. Client-side encryption

Relay Nodes never see plaintext. Compromise of any number of relays leaks only ciphertext and structural metadata (sizes, object counts). Plaintext is protected by the DEK, which never leaves the owner or (transiently) the Dispatcher and runner TEE.

24.2 10.2. Shard ID opacity

`shard_id = BLAKE3(volume_id || object_path || write_id)` is a one-way hash. Relay Nodes cannot learn object paths, detect overwrites, or enumerate a volume from shard IDs alone. This enables privacy but forces Layer-3 path enforcement to be weak; see §2.6 and §10.4.

24.3 10.3. Erasure threshold

The default 4/6 scheme tolerates any 2 simultaneous Relay Node failures per object. Three or more concurrent losses before repair completes cause data loss. Owners needing stronger durability MAY opt into 6/10, 8/12, or similar at volume creation.

24.4 10.4. Rogue runner with write token

A runner with a `WriteOnly CapToken` may write out-of-prefix shards. This is bounded by (a) the token's `max_bytes` quota, (b) orphan shard GC after `ORPHAN_SHARD_TTL`, and (c) coordinator post-commit verification. The last step is detection, not prevention: a rogue commit can be observed until a coordinator corrects it.

24.5 10.5. Relay eclipse and withholding

A minority of relays withholding a shard cannot cause data loss (K surviving shards suffice). PoR challenges detect silent withholding across the full relay set on a rolling basis. Data-withhold-

ing attacks on the manifest shard specifically are detectable via the on-chain `manifest_root` mismatch.

24.6 10.6. Sybil resistance

Relay registration requires a non-trivial `MIN_RELAY_STAKE`, and `NodeSelector` rewards diverse, healthy, reputationally clean relays. A sybil attacker must stake proportionally to the fraction of placements desired, and each sybil faces independent PoR slashing risk.

24.7 10.7. Key compromise

Loss of the cold wallet key loses control of the account (and thus the ability to issue new `DelegationCerts` or `CapTokens`), but does NOT by itself leak data: without the wrapping key, the on-chain `wrapped_dek` is not decryptable. Loss of the hot CBFS key requires revoking the `DelegationCert` on-chain (effective within the 60 s cache bound) and issuing a new one.

24.8 10.8. Dispatcher trust (v1)

In v1 the Dispatcher transiently handles plaintext DEKs during attachment sealing. This equates to the trust already placed in the Dispatcher for runner selection and `CapToken` minting. v2 routes sealing through the Secrets Manager (0x04), eliminating plaintext exposure outside the runner TEE.

24.9 10.9. Replay and nonce reuse

`DelegationCert` signatures commit to `expires_at_ms`, `chain_id`, `network`, and `aud`; owner-token signatures commit to the version byte and full payload. Control-plane requests carry `X-Cbfs-Nonce` (LRU replay cache, 60 s TTL) or `X-Cbfs-Challenge-Id` (single-use, 30 s TTL). Peer-relay RPCs carry domain-tagged digests with a 60 s timestamp window.

25 11. Parameters (Genesis Defaults)

Volumes:

`MAX_VOLUMES_PER_ACCOUNT` = 256; `MAX_VOLUME_SIZE` = 100 GiB; `MAX_OBJECTS_PER_VOLUME` = 1,000,000; `MAX_OBJECT_SIZE` = 1 GiB; `MAX_VOLUME_NAME_LENGTH` = 64 bytes; `MAX_OBJECT_PATH_LENGTH` = 512 bytes.

Erasure coding:

DEFAULT_ERASURE_K = 4; DEFAULT_ERASURE_M = 2; MAX_ERASURE_K = 16; MAX_ERASURE_M = 8.

CapTokens:

MIN_VOLUME_CAP_TOKEN_BLOCKS = 600,000 (~7 days); owner read/put TTL $\leq$ 300 s; owner mount TTL $\leq$ 14,400 s (4 h); DelegationCert expiry $\leq$ 30 days; MAX_CAP_TOKEN_BYTES = 16 KiB.

Relay Nodes:

MIN_RELAY_STAKE = governance-tunable; MAX_RELAY_HEALTH = 100 blocks; MIN_HEALTH_FOR_ASSIGNMENT = 50; RELAY_UNSTAKE_DELAY = 7,200 blocks (~24 h); REPAIR_CHECK_INTERVAL = 300 blocks; ORPHAN_SHARD_TTL = 7,200 blocks.

Proof of Retrievability:

POR_CHALLENGE_INTERVAL = 600 blocks (~2 h); POR_RESPONSE_WINDOW = 50 blocks (~10 min); POR_MISS_PENALTY = governance-tunable; POR_FRAUD_PENALTY > POR_MISS_PENALTY; RELAY_EVICTION_PENALTY = governance-tunable; POR_CHALLENGE_FEE_SHARE = 2%.

Billing:

VOLUME_CREATION_FEE = 1,000 CBY; BASE_ATTACHMENT_FEE = 100 CBY; STORAGE_FEE_PER_BYTE_PER_EPOCH = governance-tunable; TRANSFER_FEE_PER_BYTE = governance-tunable; STORAGE_FEE_BURN_RATE = 10%; STORAGE_GRACE_EPOCHS = 7,200 blocks; VOLUME_DELETE_GRACE_EPOCHS = 7,200 blocks.

Filesystem mount:

DEFAULT_SYNC_INTERVAL = 5 s; MIN_SYNC_INTERVAL = 1 s; MAX_LOCAL_CACHE_SIZE = 10 GiB.

Cache freshness (relays):

Refresh interval = 15 s; max staleness = 60 s.

26 12. v1 Scope and Phase 2

26.1 12.1. Landed in v1

- `cbfs-auth` crate, delegation setup, request signing, owner token minting.
- CLI Cowboy mode: owner auth, relay handshake verification, local token caching.
- Dual auth on `cbfs-node`: runner CapTokens (stored) and owner CapTokens (signature-verified).

- Volume lifecycle: create, soft-delete, undelete, commit manifest root.
- Autonomous two-phase repair (self-heal + dead-node replacement).
- System-level rebalance (peer-push, CAS placement replication).
- DelegationCert rotation and revocation.
- 226 unit tests across workspace, 31 SDK integration tests including network failure injection.

26.2 12.2. Deferred to Phase 2

- **Cowboy-mode FUSE mount** blocked pending owner-token refresh mechanism (unix socket sidecar, embedded refresh task, or short mount TTL plus re-auth).
- **Economic layer:** drain bonds, slashing loops, per-epoch metering settlement, full relay rewards distribution.
- **Proof of drain:** sampled Merkle-root audit of relinquished assignments.
- **Concurrent drain coordination:** destination token-bucket to prevent thundering herd.
- **Manifest-shard gap migration:** CLI to synthesize PlacementRecords for pre-placement manifests (dormant-owner volumes).
- **Auto-drain policies:** chain actor triggers drain on over-capacity, stale heartbeats, low stake.
- **Destination selection:** extend NodeSelector beyond least-used to include region / operator diversity, health, latency.
- **Observability:** Prometheus metrics, operator dashboards (CBFS core remains tracing-only).
- **v2 Secrets Manager path:** route attachment DEK sealing through 0x04 to remove Dispatcher plaintext exposure.
- **Public-volume ecosystem:** DNS-addressable actors serving static assets directly from relays per CIP-14.

26.3 12.3. Out of scope (v1)

- Content-addressed retrieval (IPFS-style).
- Alternative storage backends (S3 pass-through, local disk).
- READ_UNCOMMITTED reads.
- Cross-account volume sharing beyond CapToken grants.
- Key rotation for existing private volumes.
- Runner-initiated volume dispatch.
- Full container runtime specification.

27 13. Relationship to the Technical Whitepaper

This document layers on, but never supersedes, the Technical Whitepaper. Specifically:

- Actors, messages, timers, reentrancy, and per-handler resource limits follow Technical Whitepaper §3.
- Fee markets (Cycles, Cells) and per-block basefee adjustment follow Technical Whitepaper §4 and §17. Storage operations that touch the chain consume these meters normally.
- State rent for **on-chain actor storage** follows Technical Whitepaper §4.4 and §17.5; storage rent in this document applies only to **off-chain CBFS volumes**.
- The Runner Marketplace (Technical Whitepaper §5) is the consumer of runner CapTokens. Job dispatch integrates with the Storage Manager via `issue_attachments()` at job submission time.
- Consensus, randomness, and networking follow Technical Whitepaper §6. Relay Nodes rely on the same QC-derived randomness beacon for PoR challenge targeting.
- Governance, upgrades, and system actor hot-code replacement follow Technical Whitepaper §11 and CIP-12.
- Entitlements (Technical Whitepaper §15) MAY gate storage capabilities at the runner and actor manifest level (for example, whether an actor is permitted to open a volume at all); RAS CapTokens are the second, fine-grained layer within an entitlement's allowed scope.

Where this document and the Technical Whitepaper disagree on values or procedures, the Technical Whitepaper is the normative reference.

End of specification.